

095946- ADVANCED ALGORITHMS AND PARALLEL PROGRAMMING

Fabrizio Ferrandi

- Lecture 15 — Competitive Analysis



01 Self-organizing lists and online algorithms

02 Move-to-Front and competitive analysis

03 Potential-function proof

04 Paging: LRU vs. offline MIN



Self-organizing lists

Unsorted list L with n items

ACCESS(x) costs rankL(x): position of x, counting from 1.

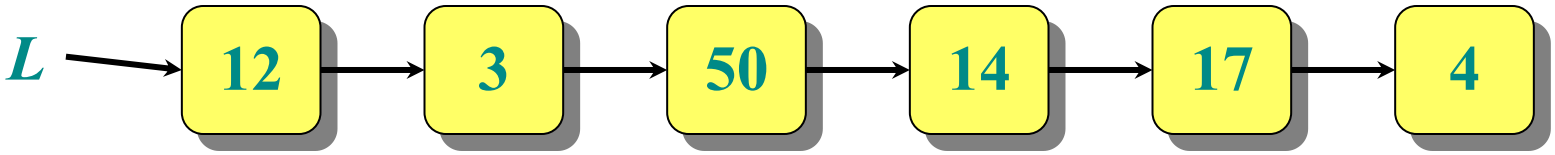
Swapping adjacent items costs 1 in the paid-exchange model.

Unsorted list L with n items

ACCESS(x) costs rankL(x): position of x, counting from 1.

Swapping adjacent items costs 1 in the paid-exchange model.

Example:

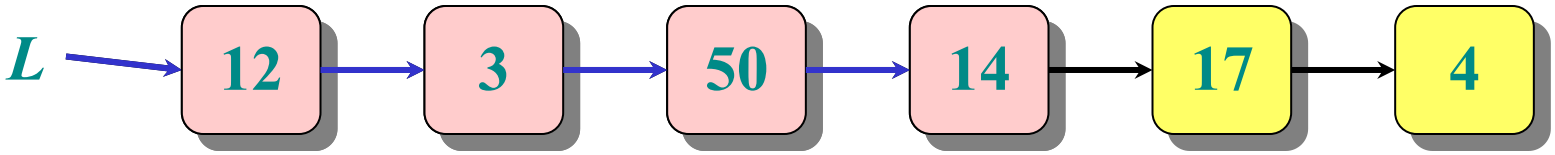


Unsorted list L with n items

ACCESS(x) costs rankL(x): position of x, counting from 1.

Swapping adjacent items costs 1 in the paid-exchange model.

Example:



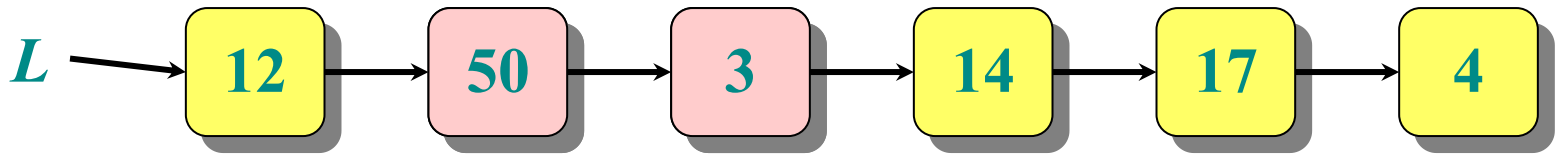
Accessing the element with key 14 costs 4.

Unsorted list L with n items

ACCESS(x) costs rankL(x): position of x, counting from 1.

Swapping adjacent items costs 1 in the paid-exchange model.

Example:

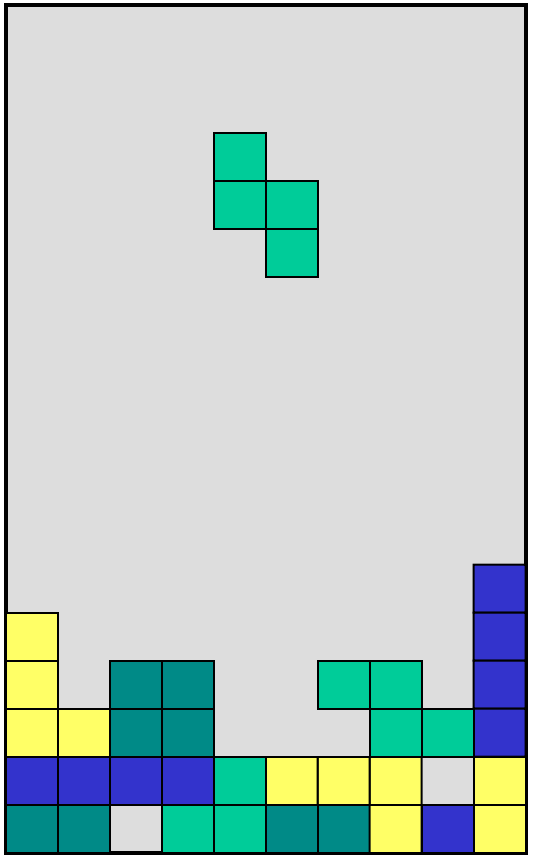


Transposing 3 and 50 costs 1.



On-line and off-line problems

Definition. A sequence S of operations is provided one at a time. For each operation, an
on-line algorithm A must execute the operation immediately without any knowledge of future operations (e.g., *Tetris*).
An *off-line* algorithm may see the whole sequence S in advance.



The game of Tetris

Goal: Minimize the total cost $C_A(S)$.



Worst-case analysis of self-organizing lists

An adversary always accesses the tail (*n*th) element of *L*. Then, for any on-line algorithm *A*, we have

$$C_A(S) = \Omega(|S| \cdot n)$$

in the worst case.

Why competitive analysis?

FROM A PESSIMISTIC BOUND TO A MEANINGFUL COMPARISON

Worst-case and average-case answers do not tell the same story.

Worst case	An adversary can repeatedly request the current tail: Ω(mn) for any deterministic online strategy.
Average case	Requires a probabilistic assumption p(x); the access sequence might instead be bursty.
Competitive analysis	Compares online A with OPT on the same access sequence, even if OPT sees the future.

OPT is a mathematical benchmark, not a practical algorithm that can predict future requests.



Average-case analysis of self-organizing lists

Assume independent requests X with P[X = x] = p(x).

For one access, with list L fixed:

$$\mathrm{E}[C_A(S)] = \sum_{x \in L} p(x) \cdot \mathrm{rank}_L(x)$$

Among fixed orderings, the minimum is achieved by sorting L in decreasing order of p(x).

Heuristic: estimate popularity using access counts.

Caution: this assumes a probabilistic model.



The move-to-front heuristic

Practice: **Implementers discovered that the *move-to-front (MTF)* heuristic empirically yields good results.**

IDEA: **After accessing x, move x to the head of L using transposes:**

$$\text{exact cost} = r + (r - 1) = 2r - 1 \leq 2r$$

The MTF heuristic responds well to locality in the access sequence S.



Competitive analysis

Definition. An on-line algorithm A is α-*competitive* if there exists a constant k such that for any sequence S of operations,

$$C_A(S) \leq \alpha \cdot C_{\mathrm{OPT}}(S) + k \ ,$$

where OPT is the optimal off-line algorithm ("God's algorithm").



What is paid for?

TWO LIST-UPDATE COST MODELS

The competitive factor depends on how we charge reordering.

PAID MODEL

$$cMTF = 2r - 1 \leq 2r$$

4-competitive

All adjacent transposes cost 1.

FREE MODEL

$$cMTF = r$$

2-competitive

Forward moves of the accessed item are free.

Sleator–Tarjan (1985), §2 and §5. This lecture proves the paid-model factor 4.



MTF is O(1)-competitive

Theorem. **MTF is 4-competitive for self-organizing lists.**



MTF is O(1)-competitive

Theorem. **MTF is 4-competitive for self-organizing lists.**

Proof setup: let L_i and L_i^* be the lists after access i.

r = rank of x in MTF before the access

r* = rank of x in OPT before the access

$c_i \le 2r$; $c_i^* = r^* + t_i$

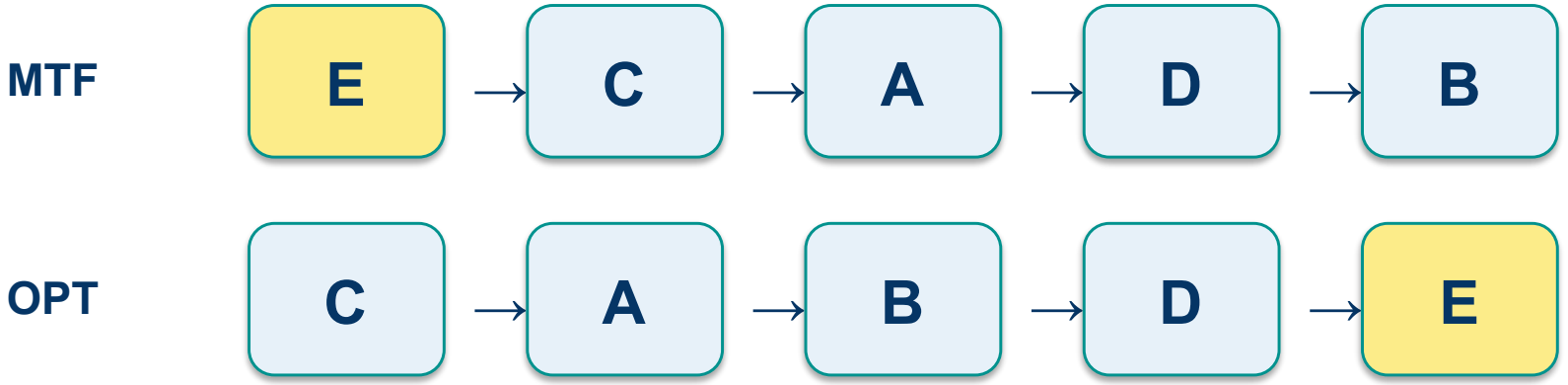
t_i counts the adjacent swaps made by OPT; MTF uses the upper bound 2r.



Why this potential?

INTUITION BEFORE THE ALGEBRA

Count the pairwise disagreements between MTF and OPT.



5 inverted pairs → Φ = 2 · 5 = 10

Potential decreases can pay for expensive MTF moves; Φ compares TWO evolving lists.



Potential function

Potential at step i: compare the order in L_i (MTF) and L_i^* (OPT).

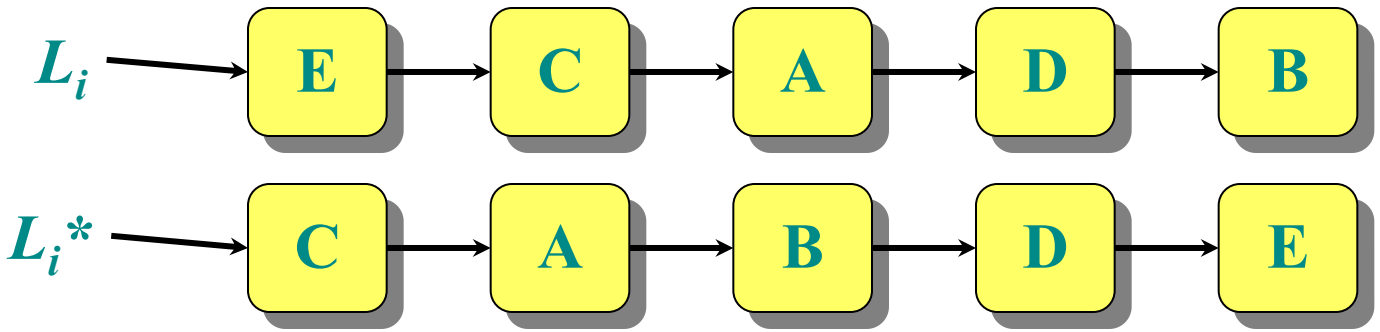
$$\begin{aligned}\Phi(L_i) &= 2 \cdot |\{(x,y) : x \prec_{L_i} y \text{ and } y \prec_{L_i^*} x\}| \\ &= 2 \cdot \#\, \textit{inversions}\,.\end{aligned}$$

Potential function

Potential at step i: compare the order in L_i (MTF) and L_i^* (OPT).

$$\begin{aligned}\Phi(L_i) &= 2 \cdot |\{(x,y) : x \prec_{L_i} y \text{ and } y \prec_{L_i^*} x\}| \\ &= 2 \cdot \#\, \textit{inversions}\,.\end{aligned}$$

Example.



$$\begin{aligned}\Phi(L_i) &= 2 \cdot |\{(E,C), (E,A), (E,D), (E,B), (D,B)\}| \\ &= 10\,.\end{aligned}$$

Reading the inversion example

WHY Φ = 10 IN THE PREVIOUS SLIDE

An inversion is an item pair whose relative order differs.

Pairs reversed between MTF and OPT:

(E,C)	**(E,A)**	**(E,B)**	**(E,D)**	**(D,B)**

5 disagreements × weight 2 = Φ = 10

The number is not a time cost: it is a measure of how far the two orderings differ.



Potential function

Potential at step i: compare the order in L_i (MTF) and L_i^* (OPT).

$$\begin{aligned}\Phi(L_i) &= 2 \cdot |\{(x,y) : x \prec_{L_i} y \text{ and } y \prec_{L_i^*} x\}| \\ &= 2 \cdot \#\, \textit{inversions}\,.\end{aligned}$$



Potential function

Potential at step i: compare the order in L_i (MTF) and L_i^* (OPT).

$$\begin{aligned}\Phi(L_i) &= 2 \cdot |\{(x, y) : x \prec_{L_i} y \text{ and } y \prec_{L_i^*} x\}| \\ &= 2 \cdot \#\, \textit{inversions}\,.\end{aligned}$$

Note that

- $\Phi(L_i) \geq 0$ **for** $i = 0, 1, \ldots,$
- $\Phi(L_0) = 0$ **if MTF and OPT start with the same list.**



Potential function

Potential at step i: compare the order in L_i (MTF) and L_i^* (OPT).

$$\Phi(L_i) = 2 \cdot |\{(x, y) : x \prec_{L_i} y \text{ and } y \prec_{L_i^*} x\}|$$
$$= 2 \cdot \#\ \textit{inversions}\ .$$

Note that

- $\Phi(L_i) \geq 0$ **for** $i = 0, 1, \ldots,$
- $\Phi(L_0) = 0$ **if MTF and OPT start with the same list.**

How much does Φ change from 1 transpose?

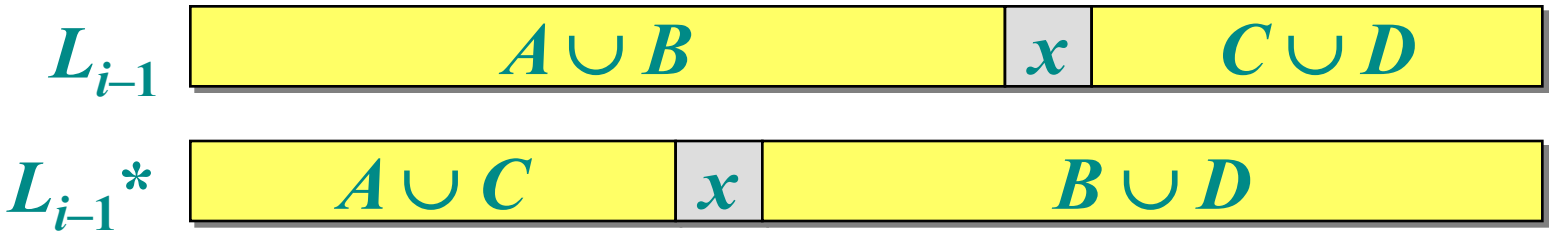
- **A transpose creates/destroys 1 inversion.**
- $\Delta\Phi = \pm 2$.



What happens on an access?

Suppose that operation i accesses element x, and define

$$A = \{y \in L_{i-1} : y \prec_{L_{i-1}} x \text{ and } y \prec_{L_{i-1}^*} x\},$$
$$B = \{y \in L_{i-1} : y \prec_{L_{i-1}} x \text{ and } y \succ_{L_{i-1}^*} x\},$$
$$C = \{y \in L_{i-1} : y \succ_{L_{i-1}} x \text{ and } y \prec_{L_{i-1}^*} x\},$$
$$D = \{y \in L_{i-1} : y \succ_{L_{i-1}} x \text{ and } y \succ_{L_{i-1}^*} x\}.$$





What happens on an access?

L_{i-1} | $A \cup B$ | x | $C \cup D$

$r = \text{rank}_{L_{i-1}}(x)$

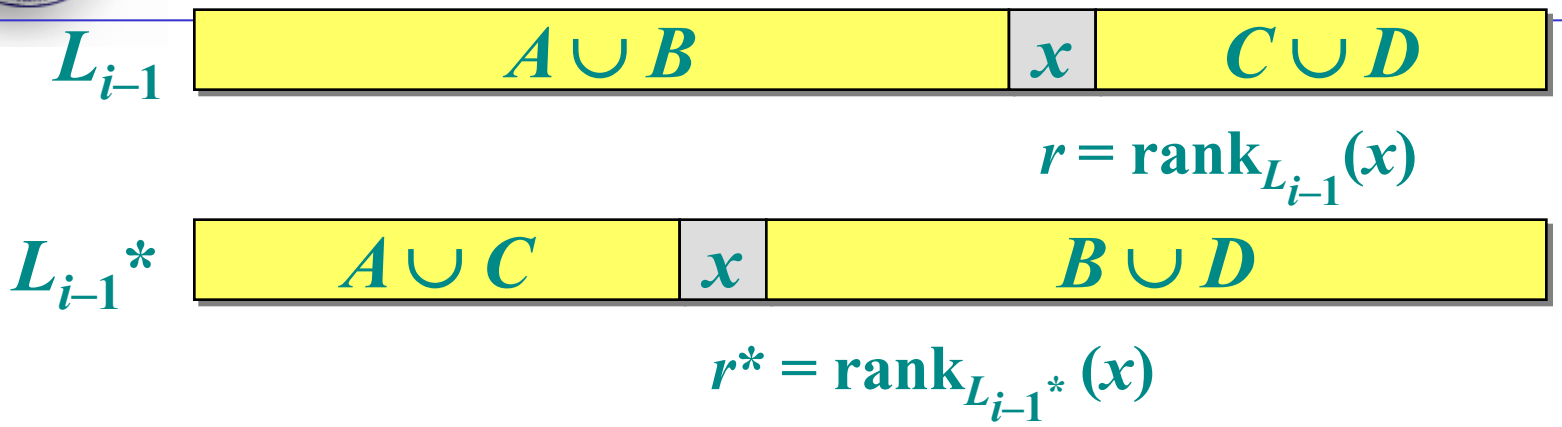
L_{i-1}^* | $A \cup C$ | x | $B \cup D$

$r^* = \text{rank}_{L_{i-1}^*}(x)$

We have $r = |A| + |B| + 1$ and $r^* = |A| + |C| + 1$.



What happens on an access?



We have $r = |A| + |B| + 1$ and $r^* = |A| + |C| + 1$.

When MTF moves x to the front, it creates $|A|$ inversions and destroys $|B|$ inversions. Each transpose by OPT creates ≤ 1 inversion. Thus, we have

$$\Phi(L_i) - \Phi(L_{i-1}) \leq 2(|A| - |B| + t_i) .$$



Amortized cost

The amortized cost for the *i*th operation of MTF with respect to Φ is

$$\hat{c}_i = c_i + \Phi(L_i) - \Phi(L_{i-1})$$

From ranks to a competitive factor

TWO SHORT ESTIMATES

Apply the potential bound and eliminate the rank r.

$$\hat{c}_i = c_i + \Delta\Phi_i \le 2r + 2(|A| - |B| + t_i)$$

Use $|B| = r - 1 - |A|$:

$$\hat{c}_i \le 4|A| + 2 + 2t_i$$

Since $|A| \le r^* - 1$, $\hat{c}_i \le 4(r^* + t_i) = 4c_i^*$

The next slide records the whole derivation in one place.



Amortized cost

The amortized cost for the *i*th operation of MTF with respect to Φ is

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi(L_i) - \Phi(L_{i-1}) \\
&\le 2r + 2(|A| - |B| + t_i) \\
&= 2r + 2(|A| - (r - 1 - |A|) + t_i) \\
&= 2r + 4|A| - 2r + 2 + 2t_i \\
&= 4|A| + 2 + 2t_i \\
&\le 4(r^* + t_i) \\
&= 4c_i^*.
\end{aligned}
$$

The grand finale

Thus, we have

$$C \sum_{i=1}^{|S|} c_i$$

$$\text{MTF}$$

$$= \sum_{i=1}^{|S|} (\hat{c}_i + \Phi(L_{i-1}) - \Phi(L_i))$$

$$\leq \left(\sum_{i=1}^{|S|} 4c_i * \right) + \Phi(L_0) - \Phi(L_{|S|})$$

$$\leq 4 \cdot C_{\text{OPT}}$$

since $\Phi(L_0) = 0$ and $\Phi(L_{|S|}) \geq 0$.



Addendum

If we count transpositions that move *x* toward the front as "free" (models splicing *x* in and out of *L* in constant time), then MTF is 2-competitive.



Addendum

If we count transpositions that move x toward the front as "free" (models splicing x in and out of L in constant time), then MTF is 2-competitive.

What if $L_0 \neq L_0^*$?

$$0 \leq \Phi_0 \leq n(n-1) = O(n^2)$$

$$CMTF(S) \leq 4 \cdot COPT(S) + \Phi_0$$

The additive term is constant w.r.t. |S| when list size n is fixed.



What the theorem does — and does not — say

INTERPRETING 4-COMPETITIVENESS

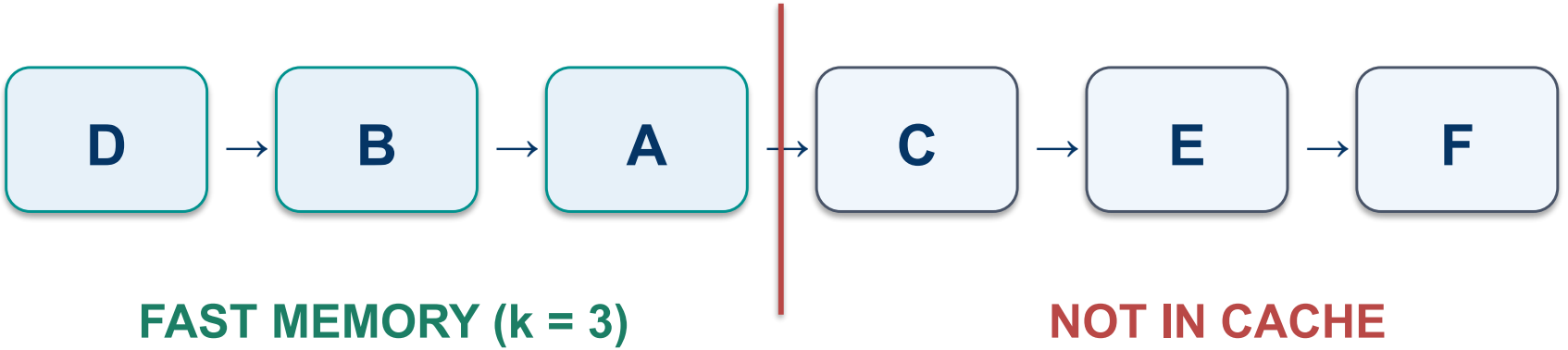
It DOES guarantee	Across every fixed access sequence, MTF total cost ≤ 4 · offline optimum (+ startup term).
It does NOT guarantee	Each operation of MTF is at most four times the same operation of OPT.
It DOES explain	Why exploiting locality can be robust even when future accesses are unknown.

Next: the very same recency idea in operating-system page replacement.

From MTF to LRU

THE SAME RECENCY PRINCIPLE

MTF maintains a recency order; LRU keeps only its first k pages.



Access to D (hit): D becomes most recent. Access to E (miss): evict A (least recent) and insert E.

Sleator–Tarjan (1985), §4: LRU paging corresponds to the recency rule of move-to-front.



Paging model and offline optimum

COST IS A PAGE FAULT, NOT A SCAN DISTANCE

$$f(r) = 0 \quad \text{for } r \le k \qquad\qquad f(r) = 1 \quad \text{for } r > k$$

LRU — online	Evict the page least recently requested. No future knowledge.
MIN — offline OPT	Evict the page whose next request is farthest in the future.

The step function f is not convex, so the earlier MTF proof cannot be reused verbatim.



Worked example: LRU versus MIN

CACHE CAPACITY k = 3 · BOTH START EMPTY

Access sequence: A → B → C → D → A → B → C → D

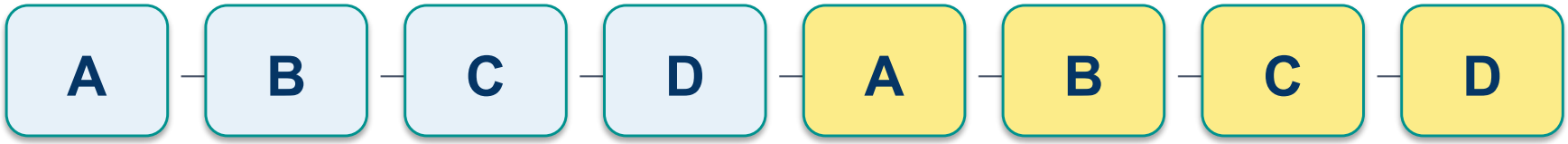
#	Request	LRU cache state (most → least recent)	LRU fault?	MIN: set	MIN
1	**A**	A	MISS	A	MISS
2	**B**	BA	MISS	AB	MISS
3	**C**	CBA	MISS	ABC	MISS
4	**D**	DCB	MISS	ABD	MISS
5	**A**	ADC	MISS	ABD	HIT
6	**B**	BAD	MISS	ABD	HIT
7	**C**	CBA	MISS	BCD	MISS
8	**D**	DCB	MISS	BCD	HIT

Total faults: LRU = 8 | MIN = 5 (MIN knows the complete future)

Why LRU can lose — and still be competitive

CYCLIC ACCESS PATTERN

Four distinct pages repeatedly compete for three cache slots.



LRU

8 misses

The just-evicted page is requested again.

MIN

5 misses

Knowledge of future requests avoids 3 misses.

This example is NOT a proof of the competitive ratio; it motivates its worst-case guarantee.



The competitive guarantee for LRU

SLEATOR–TARJAN (1985), THEOREM 6

Let LRU have k frames and offline MIN have h ≤ k frames.

$$F_{LRU}(S) \le \left[k / (k - h + 1) \right] \cdot F_{MIN}(S) + h$$

$$\text{If } h = k: \quad F_{LRU}(S) \le k \cdot F_{OPT}(S) + k$$

For k = 3 the factor is 3 (plus an additive startup term). In general this factor depends on memory size.

Reference: Sleator & Tarjan, “Amortized Efficiency…”, §4, Theorem 6, p. 207.



LRU in real systems

THE THEORY IS AN IDEALIZED MODEL

Linux memory management

Multi-Gen LRU groups pages by access recency to inform page reclamation.

docs.kernel.org/mm/multigen_lru.html

Redis in-memory cache

Approximated LRU samples eviction candidates rather than maintaining exact recency.

redis.io/docs/latest/develop/reference/eviction/

Important: real policies are variants/approximations; the theorem is about ideal LRU.

External references: official Linux and Redis documentation (consulted October 2026).



Takeaways

COMPETITIVE ANALYSIS CONNECTS THEORY AND SYSTEMS

1 **Online algorithms cannot see the future**

Evaluate them against an offline benchmark on the same request sequence.

2 **Potential captures future benefit**

Pairwise inversions yield a global proof even when single accesses are costly.

3 **Recency also governs paging**

LRU generalizes the idea, but needs a different proof and a k-dependent bound.

Question: why can LRU be costly on one sequence yet remain k-competitive?



References and further reading

Primary reference

D. D. Sleator and R. E. Tarjan, “Amortized Efficiency of List Update and Paging Rules,” Communications of the ACM 28(2), 1985, pp. 202–208.

https://www.cs.cmu.edu/~sleator/papers/amortized-efficiency.pdf

Applications and implementations

Linux Kernel Documentation — Multi-Gen LRU

https://docs.kernel.org/mm/multigen_lru.html

Redis Documentation — Key eviction / approximated LRU

https://redis.io/docs/latest/develop/reference/eviction/